

PROJECT DOCUMENTATION

ClipMind AI

Video Summarization & Key Moments Detection Platform

Submitted By

Kartik Jha

Internship Program

Infosys Springboard Internship

Internship Duration

June 29, 2026 – August 29, 2026

Acknowledgement

I would like to express my sincere gratitude to Infosys Springboard for providing me with the opportunity to work on this internship project, "ClipMind AI: Video Summarization & Key Moments Detection Platform."

This internship provided me with valuable practical exposure to the development of an AI-powered application and helped me enhance my technical knowledge and problem-solving skills. Working on this project gave me the opportunity to explore concepts related to video processing, speech-to-text transcription, AI-based summarization, and key moments detection.

I would also like to thank my mentor and everyone involved in guiding and supporting me throughout the internship. Their guidance and feedback helped me understand the project requirements and work towards the completion of the project.

Finally, I would like to acknowledge my friends and peers for their support and encouragement during the development of this project.

Kartik Jha

Abstract

Long-form video — lectures, meetings, interviews, podcasts — is easy to record and hard to revisit. Finding one relevant section usually means scrubbing through the whole recording. ClipMind AI addresses this by turning an uploaded video into a transcript, a concise summary, and a set of timestamped key moments, so a viewer can get to the relevant part in seconds rather than minutes.

Under the hood, the pipeline runs in stages: audio is extracted with FFmpeg, transcribed with Whisper (via faster-whisper), and the resulting transcript is passed to a transformer-based summarization service (BART/T5) and to separate key-moment, highlight, and keyword extraction (KeyBERT) services. Each stage depends on the one before it completing successfully, which keeps the output consistent even as videos vary in length and audio quality.

The platform includes video management, transcript handling, content analysis, key-moment identification, keywords, highlights, learning-related features, analytics, and administrative functionality. ClipMind AI demonstrates the practical integration of artificial intelligence, natural language processing, speech recognition, video processing, and full-stack web development to transform lengthy video content into structured and accessible information.

Table of Contents

Acknowledgement	2
Abstract	3
Table of Contents	4
1. Introduction	5
2. Problem Statement.....	6
3. Objectives.....	7
4. Proposed System.....	8
5. System Architecture.....	9
6. Modules and Features	10
7. Technology Stack	11
8. Methodology and Workflow	12
9. Implementation	13
10. Challenges Faced.....	15
11. Future Scope	16
12. Conclusion	17
13. References.....	18

1. Introduction

The rapid growth of digital platforms has resulted in the availability of a large amount of video content, including educational lectures, tutorials, interviews, meetings, podcasts, and other forms of long-duration media. Although video is an effective medium for sharing information, reviewing lengthy videos can be time-consuming and inefficient, especially when users are interested in finding only the most important information.

ClipMind AI: Video Summarization & Key Moments Detection Platform is an AI-powered solution designed to make video content easier to consume and analyze. The platform processes uploaded videos through video and audio processing, speech-to-text transcription, and AI/NLP-based content analysis.

The system is designed to generate concise summaries and identify important moments with relevant timestamps. It also includes keywords, highlights, learning-related features, analytics, and administrative functionality. By transforming lengthy video content into structured information, ClipMind AI aims to provide a more efficient approach to understanding and utilizing digital video resources.

2. Problem Statement

The increasing availability of long-form video content creates a challenge for users who need to quickly access and understand relevant information. Educational lectures, tutorials, interviews, podcasts, meetings, and other videos may contain valuable information, but watching an entire video to locate specific topics or important sections can be time-consuming.

Traditional video platforms generally require users to manually watch, skip, or search through videos to identify relevant content. Therefore, there is a need for an intelligent system that can analyze video content, convert spoken content into text, generate concise summaries, and identify important moments with timestamps.

ClipMind AI addresses this problem by providing an integrated platform for video management, transcription, summarization, key-moment detection, keyword extraction, highlights, and content analysis.

3. Objectives

- Build a platform for uploading, storing, and managing video content.
- Transcribe spoken content from uploaded videos using automatic speech recognition.
- Store and version transcripts so they can be reused by downstream processing steps.
- Generate concise, accurate summaries from transcripts using transformer-based NLP models.
- Detect key moments and significant segments within a video's timeline.
- Surface timestamped highlights for the most important sections.
- Extract keywords to give a quick, scannable view of each video's topics.
- Support learning workflows — bookmarks, history, and usage analytics — on top of the processed content.
- Cut the time it takes a viewer to find and understand the relevant parts of a long video.

4. Proposed System

ClipMind AI is an AI-powered video analysis platform designed to simplify the process of understanding and extracting meaningful information from long-form video content. Users upload videos, which are processed through a workflow involving media processing, speech-to-text transcription, AI/NLP analysis, summarization, and key-moment identification.

The processed outputs include the original video, transcript, AI-generated summary, keywords, highlights, and key moments with timestamps. Additional features support learning, content organization, analytics, and administration.

The proposed workflow can be represented as: *User → Authentication → Video Upload → Validation and Processing → Audio Extraction → Transcript Generation → AI/NLP Analysis → Summary → Keywords/Highlights/Key Moments → Insights and Storage → User Access.*

5. System Architecture

The architecture of ClipMind AI follows a client-server design with separate frontend and backend components. The frontend provides the user interface, while the backend manages API endpoints, business logic, media processing, AI-related services, and data operations.

The overall architecture can be represented as: *Frontend Interface* → *API Service Layer* → *FastAPI Backend* → *Routers* → *Service Layer* → *Video/AI Processing and Data Management*.

5.1 User and Application Layer

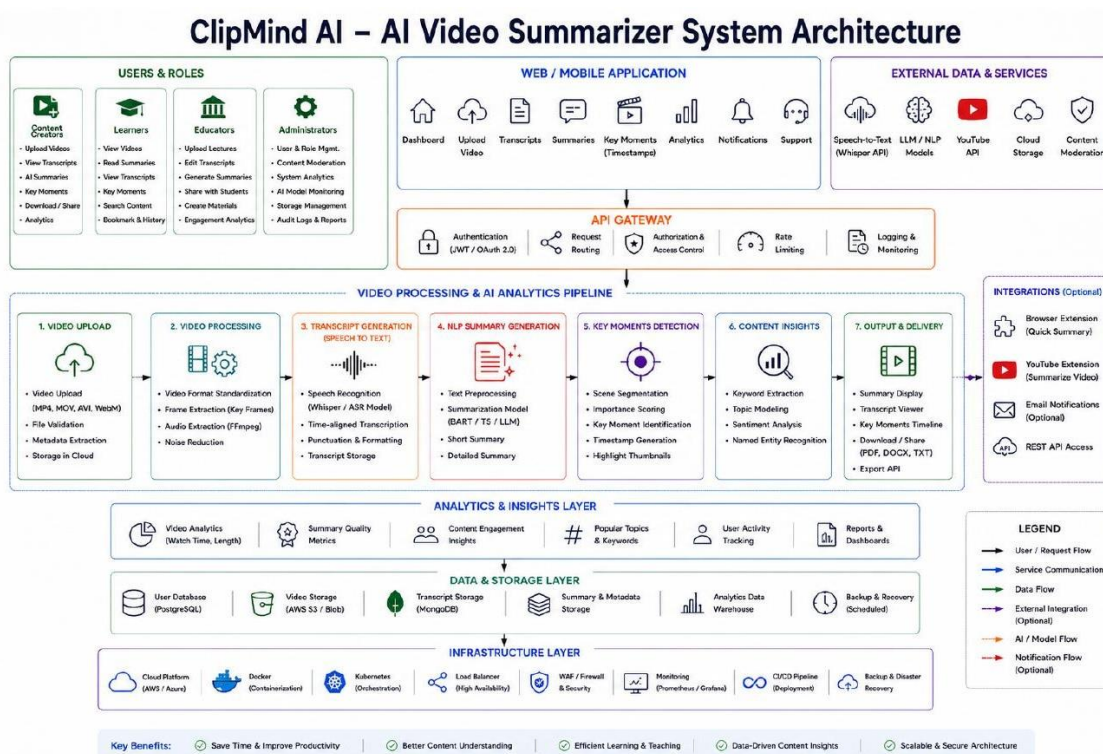
Users interact with the platform through the frontend interface, which provides pages for authentication, dashboard access, video upload, content insights, learning features, analytics, and administrative functions.

5.2 Processing Layer

Uploaded videos are handled through the video processing workflow. FFmpeg-based processing prepares media and audio for transcription. Whisper-based speech recognition generates transcripts, which are then used by summarization, keyword, highlight, and key-moment services.

5.3 Service and Data Layer

Backend routers expose feature-specific APIs, while dedicated services implement the corresponding business and processing logic. PostgreSQL, accessed through SQLAlchemy, along with core configuration and utility modules, supports application data and internal operations.



6. Modules and Features

6.1 User Management Module

Authentication, profile management, and role-based access control are supported through dedicated frontend pages, API routes, and backend services. Four user roles are defined — Content Creator, Learner, Educator, and Administrator — each with a distinct set of permissions tailored to their purpose on the platform.

6.2 Video Upload Module

Users can upload and manage videos. Uploaded content is validated, stored, and passed into the processing workflow.

6.3 Transcript Generation Module

Speech-to-text processing generates transcripts from video content and makes them available for further analysis.

6.4 Video Summarization Module

The summary service processes transcript content to generate concise video summaries using transformer-based models.

6.5 Key Moments and Highlights Module

Dedicated services identify key moments, important segments, timestamps, and highlights.

6.6 Keyword Extraction Module

Important keywords and concepts are extracted to provide a quick view of major topics.

6.7 Learning Features Module

Learning materials, learning history, and bookmarks support content organization and revisiting processed information.

6.8 Analytics and Administration Module

Dashboard, content, classroom, and system analytics are complemented by usage reports, audit logs, platform settings, user management, video management, and AI processing job monitoring.

7. Technology Stack

Category	Technologies / Components
Frontend	React.js (Vite-based), JSX component architecture
Backend	Python, FastAPI
Database	PostgreSQL, SQLAlchemy (ORM)
Speech-to-Text	Whisper, faster-whisper
Summarization	BART / T5 (transformer-based NLP models)
Keyword Extraction	KeyBERT
Media Processing	FFmpeg
Containerization	Docker
Version Control	Git and GitHub
API Communication	Frontend service layer communicating with backend routers

8. Methodology and Workflow

The project follows an end-to-end processing workflow in which raw video content is progressively transformed into structured and meaningful information.

Overall workflow: *User Authentication → Video Upload → Validation and Storage → FFmpeg Processing → Audio Extraction → Whisper Transcription → Transcript → AI/NLP Analysis → Summary → Keywords → Key Moments and Highlights → Learning Features and Insights → User Access.*

8.1 Video Upload and Validation

The user uploads a video through the application interface. The backend handles the request through the video-related API and service layers.

8.2 Media Processing

FFmpeg-related processing handles media operations and prepares content for transcription.

8.3 Transcript Generation

Whisper-based processing converts spoken audio into text, producing a transcript for further analysis.

8.4 Content Analysis

The transcript is used by dedicated services for summarization, keyword extraction, key-moment identification, and highlights.

8.5 Presentation and Access

Processed outputs are returned through API endpoints and displayed through corresponding frontend pages and services.

9. Implementation

9.1 Overall Implementation Architecture

ClipMind AI was implemented using a client-server architecture with separate frontend and backend components. The frontend communicates with the backend through an organized API service layer.

Overall flow: *React/Vite-style Frontend* → *Frontend Services* → *FastAPI Routers* → *Backend Services* → *Processing and Data Components*.

9.2 Frontend Implementation

The frontend source is organized into assets, components, layouts, pages, and services, with App.jsx and main.jsx acting as central application files. The implemented pages include Dashboard, UploadVideo, MyVideos, UploadHistory, ContentInsights, AIProcessingJobs, LearningMaterials, LearningHistory, Bookmarks, analytics pages, management pages, profile, registration, and password recovery.

9.3 Backend Implementation

The backend application is organized into core, database, models, routers, schemas, services, and utils, with main.py serving as the application entry point. This structure separates API handling, business logic, data models, validation structures, configuration, and utilities.

9.4 API Router Implementation

Feature-specific routers are implemented for authentication, users, video, transcript, summary, key moments, highlights, keywords, bookmarks, learning history, learning materials, dashboard, content analytics, classroom analytics, system analytics, administration, audit logs, and platform settings.

9.5 Service Layer Implementation

Dedicated backend services include auth_service, dashboard_service, ffmpeg_service, highlight_service, keymoment_service, keyword_service, learning_material_service, summary_service, transcript_service, video_service, and whisper_service. The service layer separates business and processing logic from the API routing layer.

9.6 Video Upload and Processing

Users upload videos through the frontend interface. The request is handled by the backend video-related API and service layer, while uploaded and processed files are managed through the backend project structure. FFmpeg-based processing supports media operations required before transcription.

9.7 Speech-to-Text and Transcript Generation

After media preparation, Whisper-based speech recognition converts spoken content into a text transcript. The transcript becomes a central input for downstream summarization and content-analysis features.

9.8 Video Summarization

The summary service and corresponding API functionality process transcript content to generate a concise representation of the video.

9.9 Key Moments, Keywords and Highlights

Separate keymoment, keyword, and highlight services support identification of important segments, extraction of relevant concepts, and generation of highlights. These outputs provide additional ways to understand and navigate processed video content.

9.10 Learning Materials and User Features

The project includes learning-material functionality together with learning history and bookmarks. Authentication and profile-related features support controlled user access and personalized interaction.

9.11 Analytics and Administration

The application includes dashboard, content, classroom, and system analytics, as well as usage reports. Administrative functionality includes user management, video management, audit logs, platform settings, and AI processing job monitoring.

9.12 Frontend–Backend Integration

Dedicated frontend service files such as `authService`, `videoService`, `transcriptService`, `summaryService`, `keyMomentService`, `highlightService`, `keywordService`, dashboard services, and analytics services communicate with corresponding backend API routes.

9.13 Docker Containerization

The backend contains a `Dockerfile` and `.dockerignore`, and Python dependencies are managed through requirements files. Containerization packages the backend application and its dependencies into a consistent execution environment. The application has since been fully deployed using Docker, with the backend running consistently inside a container.

10. Challenges Faced

10.1 Output Panel Rendering Bug in the Upload Flow

In UploadVideo.jsx, an output-panel container div was being closed immediately after it opened, so the tab content it was meant to hold ended up rendering outside the styled card instead of inside it. The bug was easy to miss visually at a glance but broke the layout structure. Tracing the JSX tree and correcting the closing tag fixed the rendering.

10.2 Enforcing the Processing Dependency Chain in the UI

The backend enforces a strict order — transcript, then summary, then key moments and highlights — but the frontend originally let users click any processing button regardless of what had completed. This let users trigger requests the backend would reject. The fix was to disable buttons for steps whose prerequisites had not finished, and to surface the backend's actual error detail in the UI instead of a generic failure message, which also made the dependency rule visible to the user rather than just enforced silently.

10.3 Managing Environment Credentials

A .env file containing real credentials was at one point included alongside the shared project files. It was removed from version control and shared archives, and the placeholder secret key was replaced with a unique value before any deployment step, to avoid shipping credentials with the codebase.

10.4 Speech-to-Text Accuracy

Whisper's transcription accuracy dropped noticeably with background noise, overlapping speakers, and strong accents in test videos, which in turn affected the quality of downstream summaries and key moments.

10.5 Processing Time for Long Videos

Transcription and summarization take noticeably longer as video length increases, which matters for how the frontend communicates progress during processing.

10.6 Docker Containerization

An early containerized build failed due to an invalid pinned dependency version — starlette==1.3.1 in requirements.txt — which did not correspond to a real published release and caused the Docker build to fail during dependency resolution. Correcting the version constraint resolved the build, and the backend now runs consistently inside the container, matching local behavior. This highlighted the importance of validating pinned dependency versions against what is actually published, rather than assuming a requirements file is correct until it has been built in a clean environment.

11. Future Scope

11.1 Improved AI Models

More advanced models can improve summarization quality, content understanding, and key-moment identification.

11.2 Multi-Language Support

Additional languages could make transcription and content analysis accessible to a wider audience.

11.3 Real-Time Processing

Future versions could explore near-real-time transcription and analysis for live lectures, meetings, or webinars.

11.4 Enhanced Key Moment Detection

Combining transcript analysis with richer audio or visual signals could improve important-segment detection.

11.5 Personalized Learning

Learning history and bookmarks could be extended into recommendation and personalized learning workflows.

11.6 Cloud Scalability

Cloud infrastructure and background processing could support larger workloads and more concurrent users.

11.7 Mobile Application

A mobile interface could extend access to uploads, transcripts, summaries, and learning features.

12. Conclusion

Building ClipMind AI meant coordinating a multi-stage AI pipeline with a full-stack web application around it — getting Whisper transcription, transformer-based summarization, and key-moment detection to run in the right order, and making sure the frontend accurately reflected what the backend was actually enforcing rather than assuming it. The dependency-chain bug and the credentials-handling issue described in Section 10 were among the most instructive parts of the project: both were the kind of gap between “the backend is correct” and “the whole system behaves correctly” that only shows up once frontend and backend are integrated and tested together, not while building either side in isolation.

The FastAPI backend and React/Vite frontend communicate through a modular router and service structure, with Docker packaging the backend for consistent deployment — a deployment that has now been completed successfully. That separation made it possible to debug and fix each of the issues above independently without destabilizing the rest of the system.

13. References

- Infosys Springboard Internship Project Specification – ClipMind AI: Video Summarization & Key Moments Detection Platform.
- OpenAI Whisper – Speech recognition and speech-to-text technology.
- FastAPI Documentation – Backend API development framework.
- FFmpeg Documentation – Video and audio processing.
- Docker Documentation – Application containerization.
- Python Documentation – Python programming language and ecosystem.
- GitHub – Source code repository and version control platform.
- ClipMind-AI Project Repository – <https://github.com/kartik05i/ClipMind-AI>